Prometheus, Grafana, and Loki – Theory Notes

---

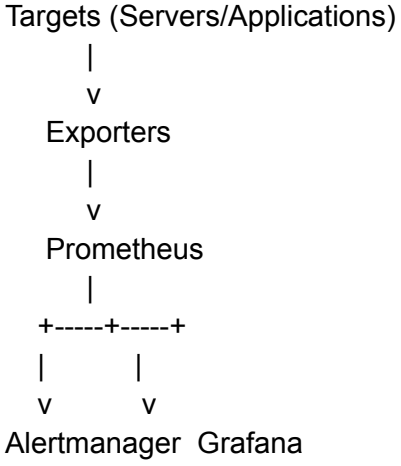
1. What is Prometheus?

Prometheus is an open-source monitoring and alerting system used to collect, store, and analyze metrics from servers, applications, containers, databases, and cloud services.

Key Features

- Time-series database
- Pull-based metric collection
- Powerful query language (PromQL)
- Alerting capabilities
- Service discovery
- Multi-dimensional data model using labels

Architecture



---

2. Prometheus Components

Component	Purpose
Prometheus Server	Collects and stores metrics
Exporters	Expose metrics in Prometheus format
Alertmanager	Handles and routes alerts
Pushgateway	Receives metrics from short-lived jobs
Grafana	Visualizes metrics

---

3. Prometheus Data Model

Metrics are stored as:

metric_name{label="value"} value

Example:

node_cpu_seconds_total{cpu="0",mode="idle"} 12345

Metric Types

Counter

- Continuously increases
- Resets only after restart

Examples:

- Total requests
- Total errors

Gauge

- Can increase or decrease

Examples:

- CPU usage

- Memory usage
- Temperature

Histogram

- Measures value distribution in buckets

Examples:

- Request duration
- Response size

Summary

- Calculates quantiles and averages

Examples:

- API latency percentiles

---

4. Exporters

Exporters expose metrics from different systems.

Common Exporters

Exporter	Purpose
Node Exporter	Linux server metrics
Blackbox Exporter	Endpoint monitoring
MySQL Exporter	MySQL metrics
Redis Exporter	Redis metrics
cAdvisor	Container metrics
kube-state-metrics	Kubernetes object metrics

---

5. Prometheus Scraping

Prometheus uses a **pull model**.

Process

Prometheus
    |
    | HTTP Request
    v
Exporter
    |
    | Metrics
    v
Prometheus Database

Advantages

- Easy service discovery
- Centralized control
- Better reliability
- Simplified security management

---

6. Labels in Prometheus

Labels add metadata to metrics.

Example:

```
http_requests_total{
  method="GET",
  status="200"
}
```

Benefits

- Flexible filtering
- Aggregation
- Grouping

- Multi-dimensional analysis

---

7. PromQL (Prometheus Query Language)

PromQL is used to query and analyze metrics stored in Prometheus.

Common Operations

Filtering
http_requests_total

Rate Calculation
rate(http_requests_total[5m])

Sum
sum(http_requests_total)

Average
avg(node_cpu_seconds_total)

Group By
sum by(instance)(http_requests_total)

---

8. Alerting in Prometheus

Prometheus can generate alerts when predefined conditions are met.

Alert Components

- Alert name
- Expression
- Duration
- Labels

- Annotations

Example Use Cases

- High CPU usage
- High memory consumption
- Disk space running low
- Application downtime
- Increased error rates

---

9. Alertmanager

Alertmanager receives alerts from Prometheus and manages notifications.

Responsibilities

- Group alerts
- Deduplicate alerts
- Silence alerts
- Route alerts
- Send notifications

Notification Channels

- Email
- Slack
- Microsoft Teams
- Telegram
- Webhooks

Flow

```
Prometheus
    |
    v
Alertmanager
    |
    +--> Email
    +--> Slack
    +--> Teams
```

---

10. What is Grafana?

Grafana is an open-source visualization and observability platform.

It helps users create dashboards and visualize metrics from multiple data sources.

Features

- Interactive dashboards
- Alerting
- Data visualization
- Multiple data sources
- Role-based access control

---

11. Grafana Architecture

Data Sources
    |
    v
  Grafana
    |
    v
Dashboards

---

12. Grafana Data Sources

Supported data sources include:

- Prometheus
- Loki
- Elasticsearch
- MySQL
- PostgreSQL
- InfluxDB

- CloudWatch

---

13. Grafana Dashboards

Dashboards contain multiple panels that visualize data.

Common Visualizations

- Time Series
- Gauge
- Bar Chart
- Pie Chart
- Table
- Heatmap

Benefits

- Centralized monitoring
- Faster troubleshooting
- Better visibility

---

14. Grafana Variables

Variables make dashboards dynamic.

Examples

- Server name
- Environment
- Application
- Region

Benefits

- Reusable dashboards
- Dynamic filtering
- Simplified maintenance

---

15. What is Loki?

Loki is an open-source log aggregation system developed by Grafana Labs.

It is designed to collect, store, and query logs efficiently.

---

16. Why Loki?

Traditional logging systems index the entire log content.

Challenges

- High storage costs
- Complex setup
- Higher resource usage

Loki Approach

- Indexes labels only
- Stores log content separately
- Lower storage requirements
- Simpler architecture

---

17. Loki Architecture

Application Logs
|
v
Promtail
|
v
Loki
|
v

Grafana

---

18. Promtail

Promtail is the log collection agent for Loki.

Responsibilities

- Read log files
- Add labels
- Send logs to Loki

Common Sources

- Linux logs
- Application logs
- Docker logs
- Kubernetes logs

---

19. Log Labels in Loki

Labels identify and organize logs.

Example:

job="nginx"
host="server1"
environment="production"

Benefits

- Faster searches
- Easier filtering
- Better organization

---

20. LogQL (Loki Query Language)

LogQL is used to query logs stored in Loki.

Capabilities

- Filter logs
- Search keywords
- Use regular expressions
- Generate metrics from logs
- Count occurrences

---

21. Metrics vs Logs

Metrics	Logs
Numeric values	Text records
Continuous monitoring	Detailed events
Low storage usage	Higher storage usage
Fast aggregation	Detailed troubleshooting

Example

Metrics:

CPU Usage = 80%
Memory Usage = 70%

Logs:

Database connection failed
User login successful
Application timeout error

---

22. Prometheus vs Loki

Feature	Prometheus	Loki
Data Type	Metrics	Logs
Storage	Time-series database	Log storage
Query Language	PromQL	LogQL
Purpose	Monitoring	Log Management
Data Format	Numeric	Text

---

23. Observability Pillars

Modern observability consists of three pillars:

Metrics

Collected by Prometheus.

Examples:

- CPU
- Memory
- Network
- Disk

Logs

Collected by Loki.

Examples:

- Error logs
- Access logs
- Application logs

Traces

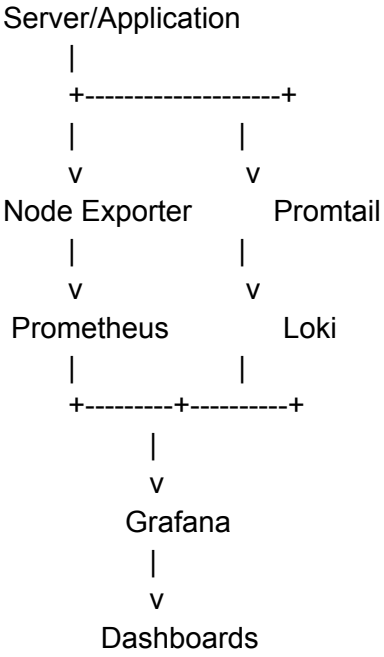
Track request flow across services.

Examples:

- API call journey
- Microservice communication

---

24. Complete Monitoring and Logging Flow



---

25. Interview Questions

Prometheus

1. What is Prometheus?
2. Explain Prometheus architecture.
3. What are exporters?

4. What is scraping?
5. Difference between Counter and Gauge?
6. What is PromQL?
7. Explain Alertmanager.
8. Pull vs Push model?

Grafana

1. What is Grafana?
2. What are data sources?
3. What are dashboard variables?
4. Difference between Grafana and Prometheus?

Loki

1. What is Loki?
2. What is Promtail?
3. Why is Loki preferred over ELK in some cases?
4. What is LogQL?
5. How does Loki store logs?

Scenario-Based Questions

1. How would you investigate high CPU usage?
2. How would you identify the cause of application downtime?
3. How would you monitor container logs?
4. How would you troubleshoot increasing error rates?
5. How do Prometheus, Grafana, and Loki work together?

One-Line Summary

Prometheus collects metrics, Loki collects logs, Grafana visualizes both, and Alertmanager sends notifications when issues occur.